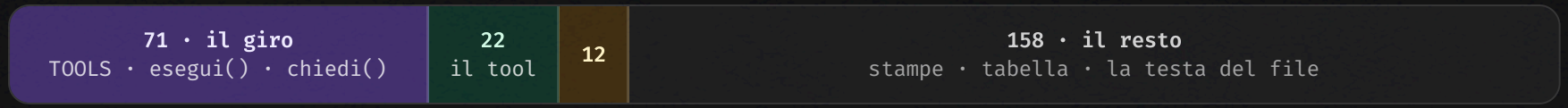
OpenAI Agents SDK: il loop gestito

Modulo 3 · Lezione 10



Il giro è la parte che si ripete

`assistente.py` della lezione scorsa: **263 righe**



`12 = le ISTRUZIONI`

`le 50 righe di chiedi() non sanno niente di Lumen; le 21 di TOOLS ed esegui() hanno la stessa forma ovunque`

- chiamare il modello con istruzioni e lista dei tool
- eseguire i tool che chiede
- rimandare i risultati con lo stesso `call_id`
- ripetere finché arriva testo
- fermarsi a un tetto

Sarebbero le stesse in qualunque altro assistente: quando un pezzo di codice è uguale ovunque, finisce in una libreria.

> Stasera è l'**OpenAI Agents SDK**. Ogni suo pezzo corrisponde a una riga che avete già scritto.

Agenda

	Blocco	Dove	Tempo
	Apertura: il giro è la parte che si ripete	slide	5′
	Setup: venv, `openai-agents`, chiave, collection	`scripts/`	5′
	Teoria con cinque dimostrazioni del docente: `Agent`, `Runner`, `function_tool` · lo schema · la traccia · `max_turns` · gli errori · il tracing	slide · `scaffolding/demo.py`	40′
	Codice spiegato: i due file affiancati, poi il lancio	`working/assistente_sdk.py`	20′
	Esercizio: `query_db` come tool (TODO 1), la strada dalla traccia (TODO 2) · poi 5′ di rientro	`working/assistente_sdk.py`	35′ + 5′
	Recap e prossima lezione	slide	10′

Un file solo, `working/assistente_sdk.py`, nella cartella `scripts/` di questa lezione. Una libreria nuova, `openai-agents`, in `requirements.txt`. Le istruzioni dell'esercizio sono in `CONSEGNA.md`.



Setup

FATE VOI — ognuno sulla propria macchina

`la cartella scripts/ di questa lezione · 5 minuti · istruzioni in CONSEGNA.md`

poi si torna alle slide:
l'installazione finisce da sola

I COMANDI

```
cd scripts
python3 -m venv .venv && source .venv/bin/activate
Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp ../../lezione-09-lab-rag-tools/scripts/.env .env
cp -r ../../lezione-09-lab-rag-tools/scripts/chroma . (se ce l'hai)
```

i path sono quelli del repo del corso: la vostra cartella della lezione scorsa può chiamarsi diversamente

`requirements.txt` porta `openai-agents`. Chi ha ancora il venv della lezione scorsa può riusarlo (`source` del suo `activate`) e installare solo `openai-agents==0.22.3`. Il `.env` è quello della lezione scorsa; se non ce l'hai più, alza la mano.

La collection dei documenti: se hai `chroma/` nella cartella `scripts/` della lezione scorsa, copiala dentro questa `scripts/` con `cp -r` (diventa `scripts/chroma/`). Se no, il primo lancio la ricostruisce da solo: due minuti.

L'assistente non si lancia ancora: il primo lancio è nel codice spiegato, dopo le demo.



I tre oggetti dell'SDK

Agent

Tiene insieme ciò che il modello vede a ogni chiamata: le **istruzioni**, la **lista dei tool**, il **modello**. Non chiama niente.

Runner

Fa il **giro**: chiama il modello, esegue i tool che chiede, rimanda i risultati, ripete finché arriva testo. `Runner.run_sync(agente, domanda)` è il nostro `chiedi()`.

function_tool

Un decoratore sopra una funzione Python: il **nome** è quello della funzione, la **descrizione** è la docstring, i **parametri** escono dalla firma e dalla sezione `Args`.

```python
@function_tool
def cerca_documenti(domanda: str) → str:
    """Cerca nei documenti aziendali di Lumen … ritorna i passaggi più pertinenti …
    Args:
        domanda: la domanda da cercare nei documenti.
    """
    ...

agente = Agent(name="Assistente Lumen", instructions=ISTRUZIONI, tools=[cerca_documenti], model=MODELLO)
result = Runner.run_sync(agente, domanda)
```

Il modello vede **nome, descrizione e parametri**: gli stessi tre pezzi di Modulo 3 · Lezione 7. Dopo il decoratore `cerca_documenti` non è più una funzione da chiamare: è un tool, lo chiama Runner. Il codice dentro resta nostro: la funzione vera da chiamare a mano è `rag.recupera`, o `tools.query_db`.



Demo 1: lo schema generato

DEMO DEL DOCENTE — guardate lo schermo

`python scaffolding/demo.py schema` · 1 minuto · nessuna chiamata al modello

si torna alle slide subito dopo

COSA GUARDARE

1 · prima la definizione della lezione scorsa: `definisci_tool(QueryDb, "query_db")`, dalla classe Pydantic

2 · poi quella dell'SDK: dal decoratore sopra `query_db(sql)`

3 · nome, descrizione, parametri: uguali. E due differenze piccole.



Cosa si è visto: lo stesso schema, da firma e docstring

la lezione scorsa · definisci_tool(QueryDb, "query_db")

```
"name": "query_db",
"description": "Esegue una query SQL … Tabelle e colonne:
prodotti(…) clienti(…) …",
"parameters": {
  "description": "Esegue una query SQL … prodotti(…) …",
← la docstring, di nuovo
  "properties": { "sql": { "title": "Sql", "type":
"string" } },
  "required": ["sql"], "title": "QueryDb" }
```

stasera · @function_tool sopra query_db(sql)

```
"name": "query_db",
"description": "Esegue una query SQL … Tabelle e colonne:
prodotti(…) clienti(…) …",
"parameters": {
  "properties": { "sql": { "description": "la query
SELECT da eseguire.", "type": "string" } },
  "required": ["sql"], "additionalProperties": false }
```

Stessi tre pezzi: nome, descrizione, parametri. È lo stesso Pydantic di Modulo 3 · Lezione 7, generato per noi.

A mano la docstring c'era due volte e `sql` non aveva una descrizione; con l'SDK una volta, e `sql` ha la riga di Args.

> La docstring non è un commento: è ciò che il modello legge. Lo schema del database di query_db deve stare lì.



La corrispondenza pezzo per pezzo

la lezione scorsa (a mano)		stasera (SDK)
`ISTRUZIONI`	→	`Agent(instructions=…)`
`definisci_tool(schema, nome) + la lista TOOLS`	→	`@function_tool sopra la funzione + tools=[…]`
`esegui() + il ciclo dentro chiedi()`	→	`Runner.run_sync(agente, domanda)`
`MAX_CHIAMATE`	→	`max_turns`
`usati, il log della strada`	→	`result.new_items, la traccia del run`

delle **71 righe**, 50 spariscono (una riga di `Runner`) e 21 cambiano forma (un decoratore e una lista)

i tool e le istruzioni restano **uguali**



Demo 2: la traccia di un run

DEMO DEL DOCENTE — guardate lo schermo

`python scaffolding/demo.py traccia · un run, 5 secondi`

si torna alle slide
subito dopo

COSA GUARDARE · la domanda doppia: «Lorenzo Rossi può montare il braccio LMX-4400 sulle scrivanie che ha ordinato?»

1 · `result.new_items`: gli item, nell'ordine in cui sono successi

2 · `result.final_output`: la risposta

3 · le chiamate al modello, una per turno; e la riga dei totali (chiamate, token, costo)

Leggere la traccia

```
result.new_items
ToolCallItem query_db({"sql":"SELECT c.nome, o.id, p.codice … WHERE c.nome =
'Lorenzo Rossi'"})
ToolCallItem cerca_documenti({"domanda":"Compatibilità e montaggio del
braccio LMX-4400 …"})
ToolCallOutputItem cliente | ordine_id | codice | prodotto | quantita
ToolCallOutputItem [listino-catalogo-004]
MessageOutputItem la risposta

2 chiamate al modello · 2.406 token in, 237 out · $0.00077

le chiamate al modello, una per turno
turno 1: output [function_call, function_call] · 570 token in, 165 out
turno 2: output [message] · 1.836 token in, 72 out
```

turno = una chiamata al modello
qui 2, non 3: i due tool chiesti nello stesso turno, eseguiti tutti e due prima di richiamarlo

`ToolCallItem`
il modello ha chiesto un tool. `raw_item` è la `function_call` dell'API, quella della lezione scorsa: `.name`, `.arguments`, `.call_id`

`ToolCallOutputItem`
il risultato del tool, in `output`

`MessageOutputItem`
la risposta; il testo è `result.final_output`

I totali (chiamate, token) sono in `result.context_wrapper.usage`: è la riga che il vostro main stampa sotto ogni traccia. Quello che la lezione scorsa era la lista `usati` oggi è nella traccia: la strada si ricava dai `ToolCallItem`. È il TODO 2.



max_turns: il tetto al giro

la lezione scorsa

```
if n_chiamate == MAX_CHIAMATE:
    return "(mi fermo: 6 chiamate senza una risposta)"
```

un testo di ripiego, e il programma va avanti

stasera

```
try:
    result = Runner.run_sync(agente, domanda,
max_turns=MAX_TURNI)
except MaxTurnsExceeded:
    print("(mi fermo …)")
```

un'eccezione, e il nostro codice decide cosa fare (`MaxTurnsExceeded` si importa da `agents`, come `Agent` e `Runner`)

default 10 turni

senza `max_turns` il giro si ferma a dieci turni

un turno = una chiamata al modello

due tool nello stesso turno contano uno: il tetto conta i turni, non i tool eseguiti

`MAX_TURNI = 10`

uguale al default: il file lo scrive per vederlo, e per poterlo cambiare



Demo 3: il tetto

DEMO DEL DOCENTE — guardate lo schermo

`python scaffolding/demo.py tetto · un run`

si torna alle slide subito dopo

COSA GUARDARE · «Quanti ordini ha fatto Giulia Moretti?» con max_turns=1

1 · con un turno il modello può solo rispondere; siccome chiede `query_db`, il giro non può finire: `MaxTurnsExceeded: Max turns (1) exceeded`

2 · sotto, i due item eseguiti prima di fermarsi (stanno in `e.run_data`: li legge la demo, e li conta anche il vostro main): il tool **è stato eseguito**, è la seconda chiamata che manca

> Con `max_turns=2` le quattro domande della lezione scorsa passano (`python scaffolding/demo.py tetto 2`): Lorenzo Rossi chiede i due tool in un turno solo. Se li chiedesse in due turni sarebbero 3 (un tool, l'altro, la risposta) e con 2 si fermerebbe. Il file tiene **10**, il default: lascia spazio a Luna quando cerca di nuovo. La lezione scorsa il tetto era 6.



Demo 4: un tool che si rompe

DEMO DEL DOCENTE — guardate lo schermo

`python scaffolding/demo.py rotto · due run`

si torna alle slide subito dopo

COSA GUARDARE · «Quanto costa la LMX-4400?» a un query_db fatto male apposta

senza il `try/except` di Modulo 3 · Lezione 8, e con `prezzo_listino` nello schema al posto di `prezzo`: SQLite solleva `no such column`

1 · **di default**: il testo che torna al modello, e cosa fa il modello

2 · **`failure_error_function=None`**: l'eccezione che esce dal run



Cosa si è visto: gli errori dei tool

`1 · di default`

```
An error occurred while running the tool. Please try again. Error: no such column: prezzo_listino
```

torna al modello come testo; il programma non muore, e il modello fa quello che può. In tre giri, sempre 3 chiamate:

· due volte ha riscritto la query con `SELECT *` e ha risposto 89 euro

· una volta ha riprovato la stessa query e ha detto «non disponibile»: sono le `ISTRUZIONI` («se un tool ritorna un errore due volte di fila, fermati»)

`2 · failure_error_function=None`

```
UserError: Error running tool query_db_rotto_senza_rete: no such column: prezzo_listino
```

l'eccezione esce dal run: il programma muore, a meno di un `try` nostro. Si scrive sul decoratore: `@function_tool(failure_error_function=None)`.

A Modulo 3 · Lezione 8 lo facevamo noi: `except Exception as e: return f"Errore: {e}"`, con il nostro messaggio. Il nostro `tools.query_db` lo fa ancora: stasera non cambia niente.

> Un tool rotto **non ferma il giro**: il modello riceve un testo e va avanti come può. Se lo si vuole fermare, si dice.



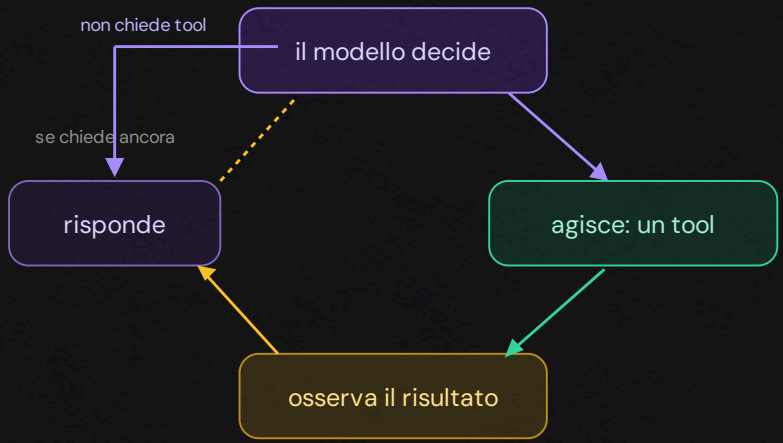
Cosa l’SDK non fa

1. I **tool restano codice nostro**: dentro `cerca_documenti` c'è `rag.recupera`, dentro `query_db` c'è `tools.query_db`, K resta 4. Il decoratore li incarta, non li sostituisce.
2. Le **istruzioni** restano le stesse, parola per parola.
3. Il **modello sceglie come prima**: nome, descrizione e parametri dei tool, la domanda, le istruzioni. Se sbaglia strada, l'SDK non c'entra: le leve sono quelle della lezione scorsa.
4. Il **rifiuto** (la domanda su Padova) è ancora questione di istruzioni.

> L'SDK fa il giro e ce ne dà la traccia.



Perché si chiama Agent



Il giro va avanti finché il modello smette di chiedere tool: legge, decide, agisce con un tool, osserva il risultato, decide ancora. Un programma che fa questo si chiama **agente**. La classe si chiama `Agent` per questo.

È un **nome**, non una tecnologia nuova: l'assistente della lezione scorsa era già un agente, solo che il giro lo scrivevamo noi.

quando serve un agente, quando basta un flusso fisso, la teoria dietro → Modulo 4 · Lezione 4

Demo 5: il tracing

DEMO DEL DOCENTE — guardate lo schermo

`python scaffolding/demo.py tracing` · un run, poi il browser

si torna alle slide subito dopo

COSA GUARDARE

1 · sotto la risposta, un `trace_id` e un link: il docente lo apre nel browser, con l'account del corso

2 · nella dashboard: l'agente, ogni chiamata al modello con input e output completi, ogni tool con argomenti e risultato, i tempi

3 · è la stessa traccia di `new_items`, con dentro anche i testi che passano, vista da fuori



Cosa si è visto: il tracing

`cosa esce`

Di default **ogni run** manda la traccia ai server di OpenAI, sotto l'account della chiave: le domande, i risultati dei tool, le risposte.

`dove`

`platform.openai.com/traces`

un run = una traccia, con il suo `trace_id`

`come si spegne`

```
OPENAI_AGENTS_DISABLE_TRACING=1
```

nel `.env`: il `trace_id` diventa no-op, non parte niente. La riga è in `.env.example`; il `.env` copiato dalla lezione scorsa non ce l'ha.

> Anche i vostri run finiscono lì, sull'account della chiave del corso. Stasera non c'è niente da fare: resta acceso. Quando un giorno i dati non dovranno uscire, si spegne.



Il file: working/assistente_sdk.py

```
MODELLO — gpt-5.6-luna · K — 4 · MAX_TURNI — il tetto al giro (10)
DOMANDE — quattro, con la strada attesa · SOLO = "…" per una domanda sola, SOLO = None per tutte e quattro
cerca_documenti — con @function_tool, già scritto: la docstring è la descrizione
TODO 1 · query_db — nella stessa forma; la funzione vera è tools.query_db (scaffolding/aikit/tools.py)
ISTRUZIONI — quelle della lezione scorsa · agente = Agent(name, instructions, tools=[…], model)
TODO 2 · strada(result) — dai ToolCallItem di result.new_items
stampa_traccia(result) · stampa_tabella(esiti)
main — Runner.run_sync(agente, domanda, max_turns=MAX_TURNI) per ogni domanda, poi la tabella attesa / presa
```

Stessa forma di `assistente.py`. Si continua nello stesso file la prossima lezione.



Codice spiegato: la migrazione affiancata

CODICE SPIEGATO — il docente spiega, voi seguite sul vostro file

`working/assistente_sdk.py` · tutto già scritto tranne i TODO · 20 minuti

si rientra dopo il lancio, con l'output a schermo

IL DOCENTE

I due file affiancati: la versione finita di `assistente.py` della lezione scorsa a sinistra, `working/assistente_sdk.py` a destra, dall'alto in basso. Poi il lancio, così com'è, con un tool solo:

```
python working/assistente_sdk.py
```

VOI

Il vostro `working/assistente_sdk.py` aperto, e il vostro `assistente.py` della lezione scorsa accanto, se ci sta. Dopo il docente, lanciate anche voi: al primo lancio senza `chroma/` indicizza, due minuti.



Cosa sparisce, cosa resta

la lezione scorsa		stasera
`class CercaDocumenti(BaseModel) + cerca_documenti()`	→	solo la funzione, con `@function_tool` e la docstring sopra
`TOOLS + esegui() (21 righe)`	→	cambiano forma: il decoratore e `tools=[…]` dentro `Agent`
`chiedi() con il ciclo (50 righe)`	→	sparisce: `Runner.run_sync(agente, domanda, max_turns=MAX_TURNI)`
`usati + le stampe dentro il ciclo`	→	`strada(result)` e `stampa_traccia(result)`, dopo il run
`ISTRUZIONI`	=	identiche, passate a `instructions`

IL LANCIO, CON UN TOOL SOLO: LA BASELINE

· i resi: la citazione, e nella traccia `tool cerca_documenti`

· Giulia Moretti: nessun tool, «lo strumento necessario non è disponibile»

· Lorenzo Rossi: cerca due volte nei documenti, e dice che gli ordini non li trova

· Padova: rifiuto senza tool, come sempre

· la tabella dice (`TODO 2`) su ogni riga

A MANO CONTRO SDK · le quattro domande, tre giri ciascuna: 12 run

· strada giusta **12/12** a mano, **12/12** con l'SDK

· stesse chiamate al modello · stessi tempi (3–4 secondi)

· meno token in input: **1.430** contro 1.811 in media (la docstring una volta sola)



⌨ Esercizio: query_db come tool, la strada dalla traccia

SCRIVETE VOI — ognuno nel proprio file

`working/assistente_sdk.py` · TODO 1 e TODO 2 · 35 minuti · istruzioni in `CONSEGNA.md`

poi 5' di rientro sulle slide, con la soluzione

TODO 1 · query_db come tool (20')

Nella forma di `cerca_documenti`, che è appena sopra: `@function_tool`; la docstring: **tutta quella di QueryDb** in `scaffolding/aikit/tools.py` (la frase su SELECT e lo schema delle quattro tabelle) più `Args` con `sql`; dentro `return tools.query_db(sql)`. Poi il tool in `tools=[…]` dell'agente.

FATTO QUANDO · Giulia Moretti in SOLO risponde 13, e nella traccia c'è `tool query_db`

TODO 2 · la strada dalla traccia (15')

`strada(result)`: dai `ToolCallItem` di `result.new_items` ai nomi dei tool chiamati (`item.raw_item.name`), poi la stessa logica della lezione scorsa: documenti / dati / documenti+dati / nessuna.

FATTO QUANDO · con SOLO = None la tabella dice **4/4**, e la riga `strada:` coincide con i tool nella traccia. Nelle nostre misure (quattro domande per tre giri) è 12 su 12; se una riga dice NO, rilanciala in SOLO due volte prima di cercare l'errore nel tuo codice.

Se finisci prima: una domanda della tua lista `LE_MIE_QUERY` (`working/debug.py` di Modulo 3 · Lezione 6) in SOLO, e guarda nella traccia se Luna cerca una volta o due; `print(query_db.params_json_schema)` subito sotto `query_db`, per vedere ciò che il modello legge; un terzo tool con `@function_tool` a partire da `tools.prezzo_prodotto`.



Rientro: la soluzione dei due TODO

```python
@function_tool
def query_db(sql: str) -> str:
    """Esegue una query SQL (solo SELECT, dialetto SQLite) sul database degli
    ordini Lumen e ritorna le righe trovate. Tabelle e colonne:
    prodotti(codice TEXT, nome TEXT, categoria TEXT, prezzo REAL)
    …                                          # le altre tre righe di QueryDb
    Args:
        sql: la query SELECT da eseguire.
    """
    return tools.query_db(sql)

agente = Agent(name="Assistente Lumen", instructions=ISTRUZIONI, tools=[cerca_documenti, query_db], model=MODELLO)

def strada(result):
    usati = [item.raw_item.name for item in result.new_items if isinstance(item, ToolCallItem)]
    ...                                        # poi le quattro righe della lezione scorsa
```

La docstring è ciò che il modello legge: lo schema ci deve stare, o le query escono con colonne che non esistono.

`.name` ce l'hanno solo i `ToolCallItem`: gli output e la risposta no.

Nella traccia compare «An error occurred … `TODO 1`»? Il tool è cablato e dentro c'è ancora lo stub: è la Demo 4 dal vivo.

> Chi non ha finito lo ricopia adesso: la prossima lezione parte da questo file.



Recap

1. Il giro è **uguale in ogni progetto**: da stasera lo fa `Runner`.

2. Un tool è una **funzione con una docstring**: `@function_tool` ne fa nome, descrizione e parametri, ed è tutto ciò che il modello vede.

3. Il run ritorna una **traccia**: `new_items` dice quali tool ha chiamato e con cosa, `usage` quanto è costato, `max_turns` quando fermarsi. Un tool rotto non ferma il giro.

4. L'SDK **non sceglie e non rifiuta**: i tool, le istruzioni e le leve restano nostri.



La prossima lezione

stasera
il giro lo fa l'SDK; ogni run parte da zero

→

Modulo 3 · Lezione 11: le sessioni dell'SDK
la storia la tiene una sessione e si ripassa al modello a ogni run; i dialoghi di Modulo 3 · Lezione 3 rifatti; stesso file

un follow-up, oggi

`«e per il secondo caso?»`

senza storia il modello cerca letteralmente «secondo caso»

Poi, a Modulo 3 · Lezione 12, ognuno porta il proprio assistente sull'SDK.



Grazie

prossima lezione: Modulo 3 · Lezione 11

